



PRELIM PROJECT AI POWERED HOME VIRTUAL ASSISTANT REPORT

Submitted by:

de Dios, Louise Jeanne T.
Mangubat, Angel Julliane I.

Submitted to:

Mr. Roberto L. Malitao

In Partial Fulfillment of the Requirements for the Course

BSCS 3112 Artificial Intelligence – 9420-AY126

Degree Program

Bachelor of Science in Computer Science with Specialization in Data
Science

August 17, 2026



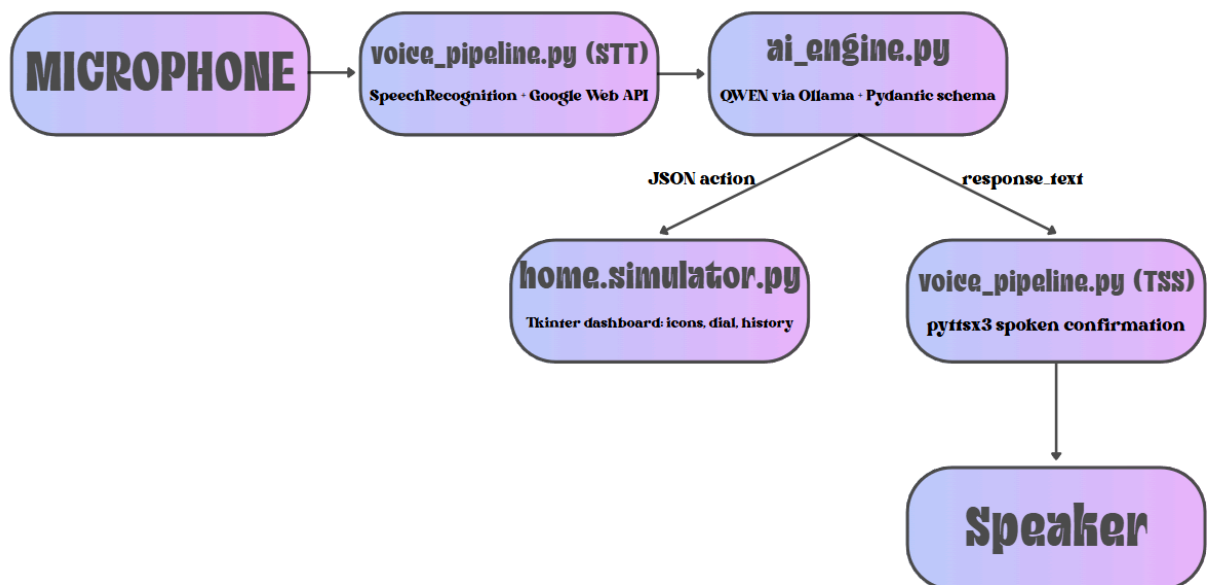
Introduction

This report documents the preliminary build of an AI-powered home virtual assistant. The system accepts spoken commands, interprets user intent with a locally-hosted language model, and reflects the resulting actions on a simulated smart-home dashboard with a synchronized spoken confirmation. Development was split across two modules built in parallel and integrated on a single machine: the AI/voice pipeline and the GUI dashboard. All inference runs locally via Ollama; the one external dependency is Google's Web Speech API for transcription (see section 4 for the offline-operation gap this creates).

System Architecture

The assistant runs the GUI on the main thread and the voice loop on a background daemon thread. A threading.Event lets the Pause/Resume button stop the microphone loop without blocking the GUI. Recognized speech is sent to a local QWEN model via Ollama, which returns a structured JSON action validated against a Pydantic schema; the GUI and a spoken confirmation are updated from that action.

main.py - GUI (main thread) - Voice Loop (background daemon thread), threading Event pause/resume

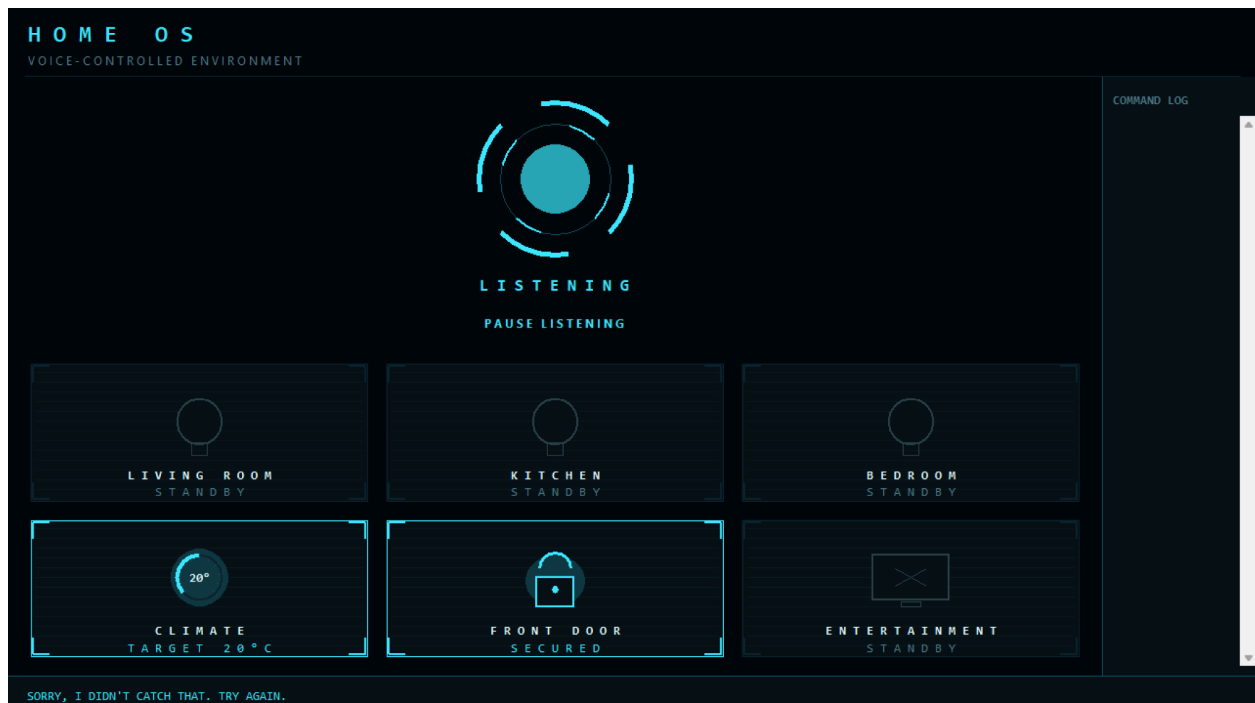




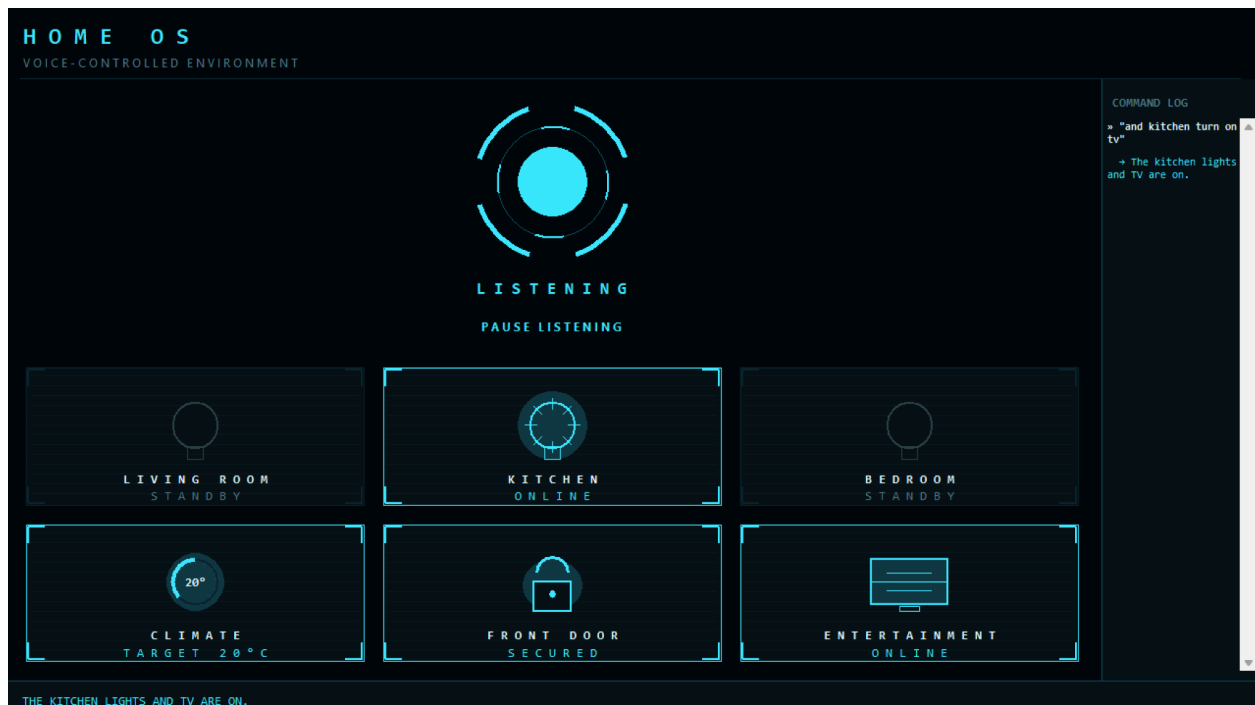
GUI Screenshots & Resource Usage

Command 1: Turn on the TV and Kitchen Light

BEFORE



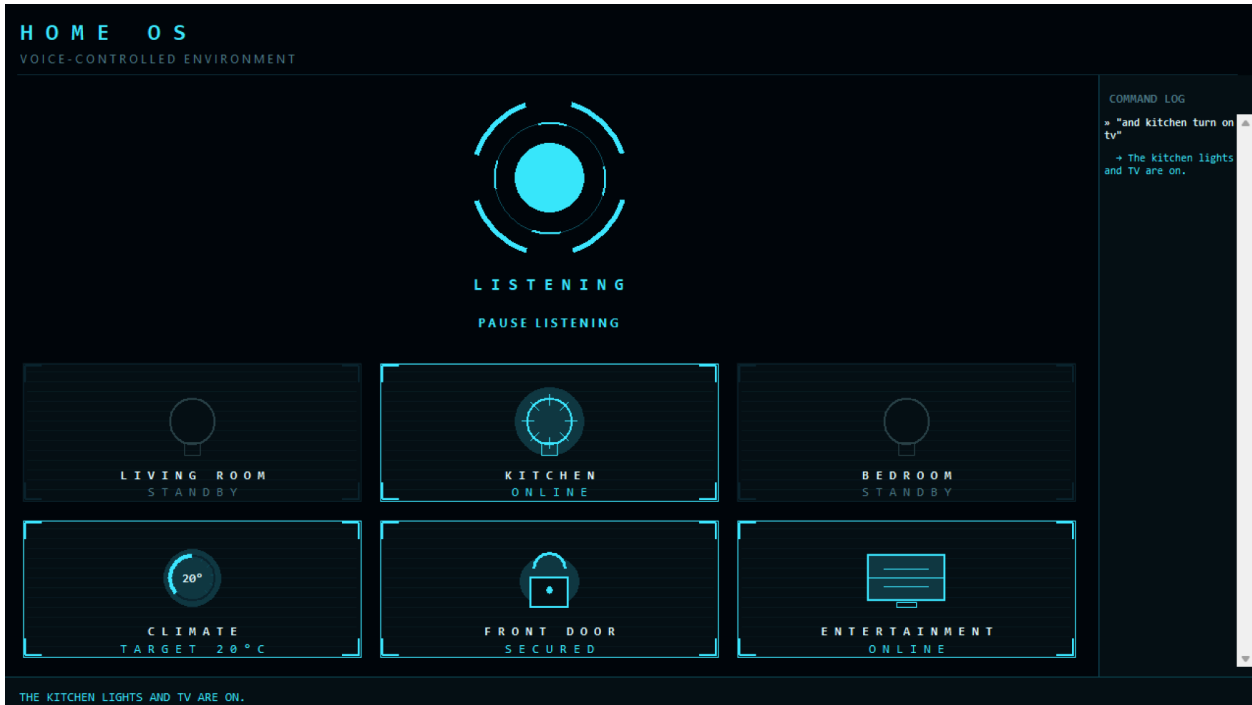
AFTER



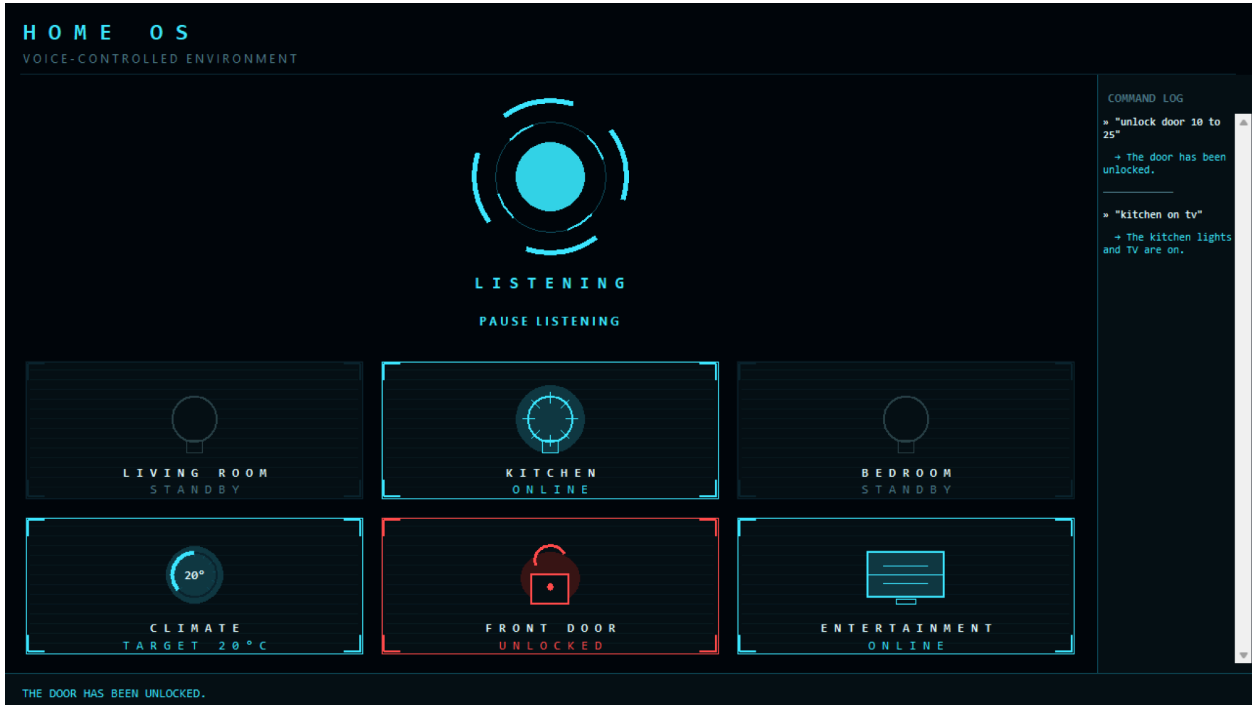


Command 2: Unlock the Front Door

BEFORE



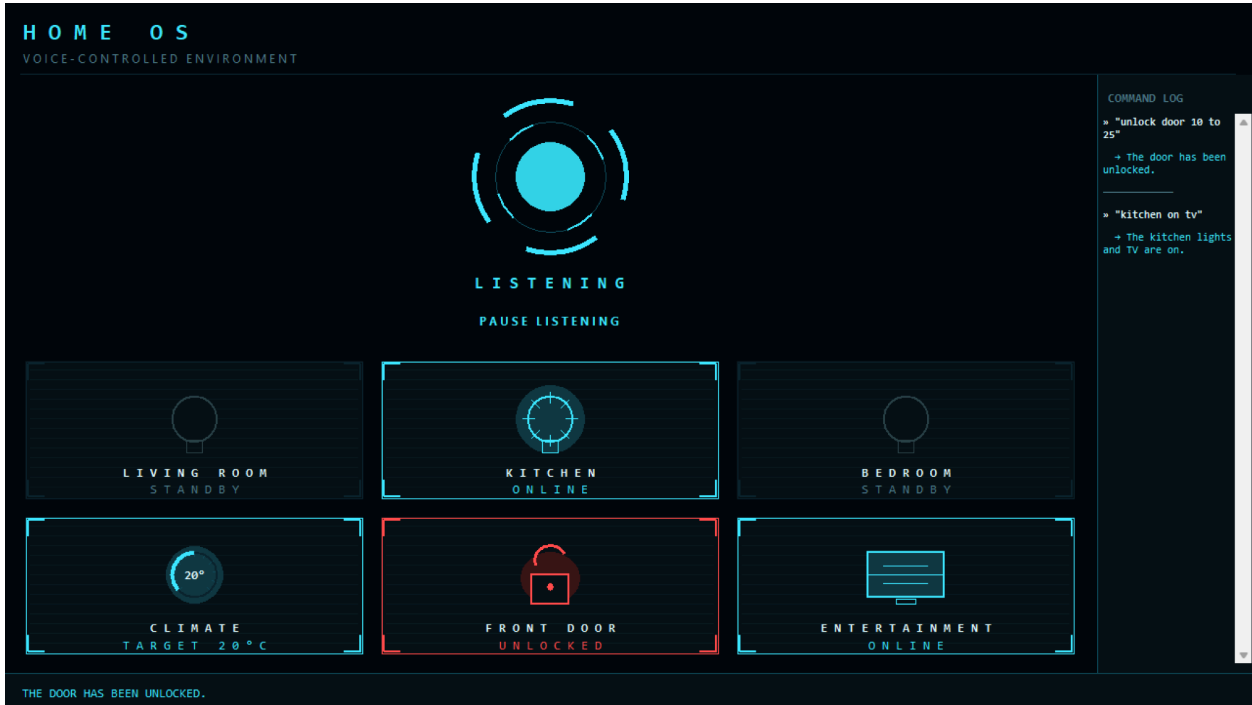
AFTER



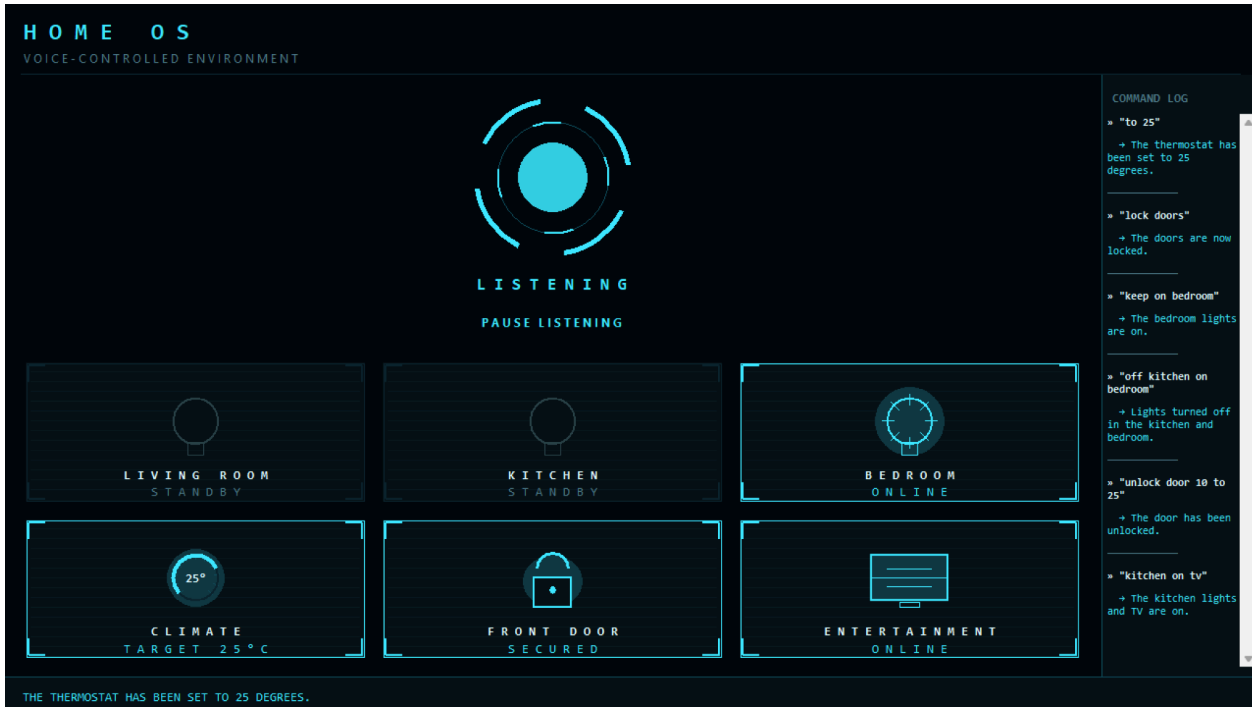


Command 3: Turn off the Kitchen Light, Open the Lights at the Bedroom and Lock the Front Door

BEFORE



AFTER





Peak CPU/RAM Metrics

Measured with monitor_resources.py (psutil) while each command below ran end-to-end (mic capture through GUI update).

Voice Command	Peak CPU (%)	Peak Ram (MB)	Duration (s)
Turn on the TV and Kitchen Light	0%	4.1 MB	12.7 s
Unlock the Front Door	0.2%	4.1 MB	22.0 s
Turn off the Kitchen Light, Open the Lights at the Bedroom and Lock the Front Door	0.5%	4.1 MB	35.9 s

The measured results show **low CPU and RAM utilization**, with peak CPU usage ranging from 0% to 0.5% and peak RAM remaining at approximately 4.1 MB. However, command duration increases as the number of requested actions increases, with the longest multi-action command taking 35.9 seconds. This supports the identified need for further profiling and optimization of the STT and local Qwen inference stages.

Pair-Programming Task Division Matrix

Task	Owner	Status	Notes
ai_engine.py (Qwen prompt, Pydantic schema, JSON parsing)	Angel	Done	Hallucination Fixed by Louise
voice_pipeline.py (STT + TTS)	Angel	Done	bug fixed by Louise
home_simulator.py (Tkinter GUI base + icons/dial/history)	Angel	Done	—
main.py (threading, pause/resume wiring)	Angel	Done	—
Offline STT exploration (vosk / pocketsphinx)	Louise	In progress	—
GUI polish (transition animation, processing state)	Louise	In progress	—
Preliminary Project Report	Louise	In progress	This document